**Ali Shavakhi**

**Project Report**

Given the availability of extensive labeled datasets for this project, I decided to train a pre-trained YOLO v11 model on these datasets. However, there was a key challenge: adapting the datasets to the specific format required by YOLO.

**Dataset Preparation**

YOLO accepts datasets in a specific normalized format, which is detailed in its official documentation. To address this requirement:

1. **Dataset Normalization**:
    - I wrote separate programs for each of the two datasets to convert their labels into YOLO's standard format. These include:
        - **traffic-light-set-labelizer.py**: This script processes the Bosch Small Traffic Lights Dataset, normalized according to YOLO's requirements. Key functions include:
            - Parsing the YAML file for traffic light annotations.
            - Correcting bounding box coordinates and clipping them within image boundaries.
            - Converting the data into YOLO-compatible text files for each image.
        - **traffic-sign-set-labelizer.py**: This script processes traffic sign datasets and generates YOLO-compatible labels. It:
            - Reads annotations from a structured text file.
            - Handles and excludes images without valid labels or those marked as miscellaneous.
            - Extracts bounding box details, calculates normalized center coordinates, width, and height.
            - Writes these details into YOLO format for each image.
2. **Class Mapping**:
    - I created a **classes.py** file to define the mapping of class names to indices required by YOLO. This file contains:
        - A dictionary mapping class names (e.g., STOP, GIVE_WAY) to unique numeric indices.
        - A reverse dictionary to map indices back to their corresponding class names for easier visualization and debugging.

3. **Dataset Merging**:
    - After normalizing the datasets, I merged them into a single unified dataset for further processing.

**Uploading to Roboflow**

Since my local computer was not capable of handling the training process, I decided to use the Roboflow platform. I uploaded the unified dataset to Roboflow, which took approximately 8 hours to complete.

**Preprocessing and Augmentation**

On Roboflow:

- I applied various preprocessing techniques and augmentations to enhance the dataset. These steps included:
    - Resizing images to match YOLO's input dimensions.
    - Applying random flips and rotations to increase dataset diversity.
    - Adding brightness and contrast adjustments to improve model robustness.
    -

**Training the Model**

1. **Training on Roboflow**:
    - I attempted to train the YOLO model directly on Roboflow. However, the platform did not produce a usable output model after training.
    - I obtained two visual outputs from the training on Roboflow, which I will attach as **Photo 1** and **Photo 2** at the end of this paper.
2. **Training Locally**:
    - I wrote a new script named train.py to perform the training process locally. While this approach is feasible, it requires significant computational resources, and training took approximately one day to complete.
    - The train.py script was designed to handle both "hard" and "easy" datasets effectively by dynamically adjusting hyperparameters and batch sizes as needed.
3. **Training on Google Colab**:
    - To overcome the issue, I created a Colab notebook for training the YOLO model. The notebook successfully produced an output model after training.
    - I will share the link to this Colab notebook for reference.

**Testing and Visualization**

After training, I created a script named predict.py to test the trained model on test datasets and visualize the results. This script:

- Utilizes the Roboflow API to load the trained model.
- Reads images from test directories, processes them, and applies the trained model for prediction.
- Draws bounding boxes and class labels on detected objects within the images.
- Saves the processed images with predictions to an output directory for review.

**Next Steps**

- I plan to test the trained model's performance and evaluate its accuracy on unseen data.
- Based on the results, I will fine-tune the model if necessary.

**The results of Roboflow**

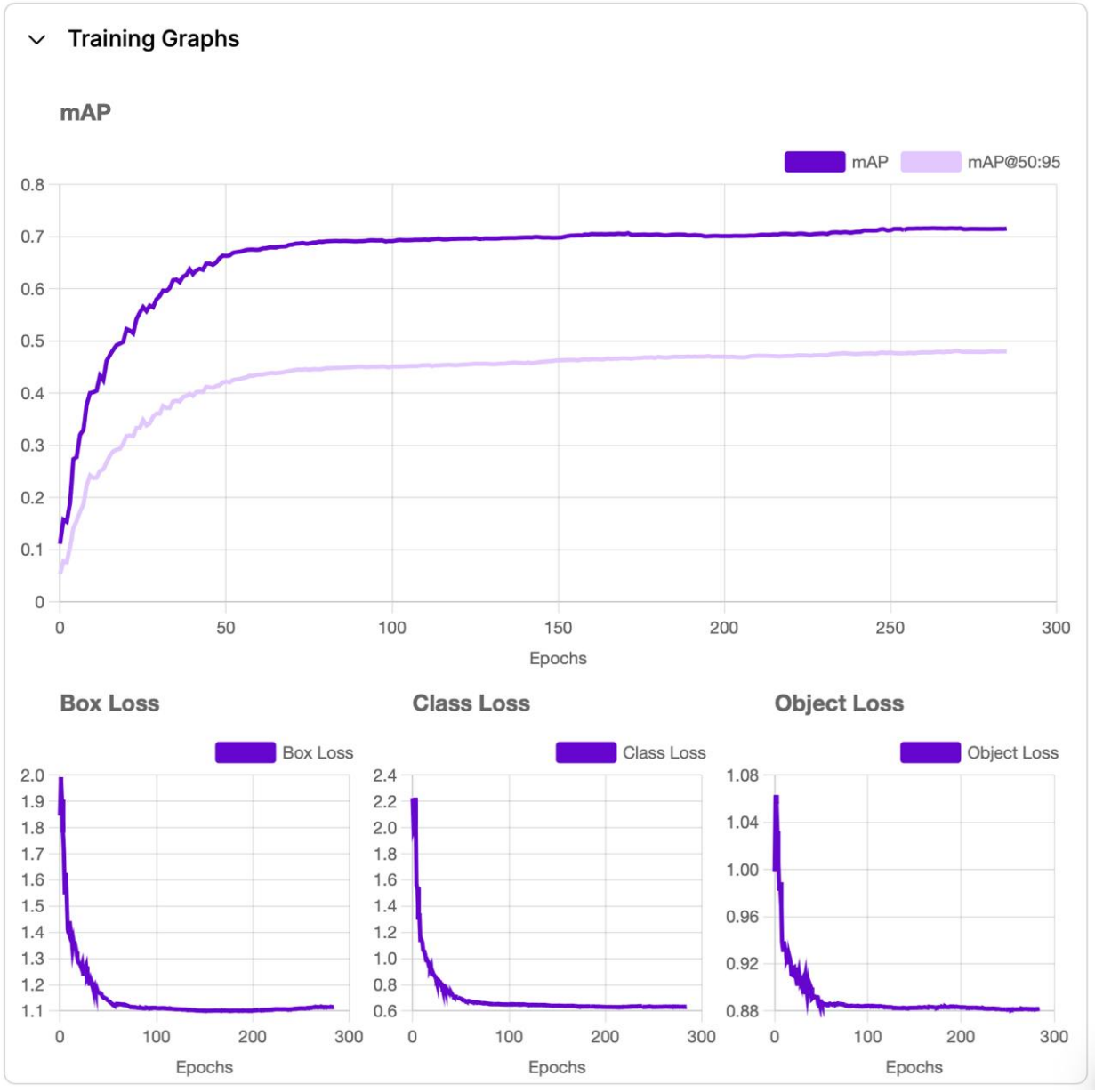



Photo 1

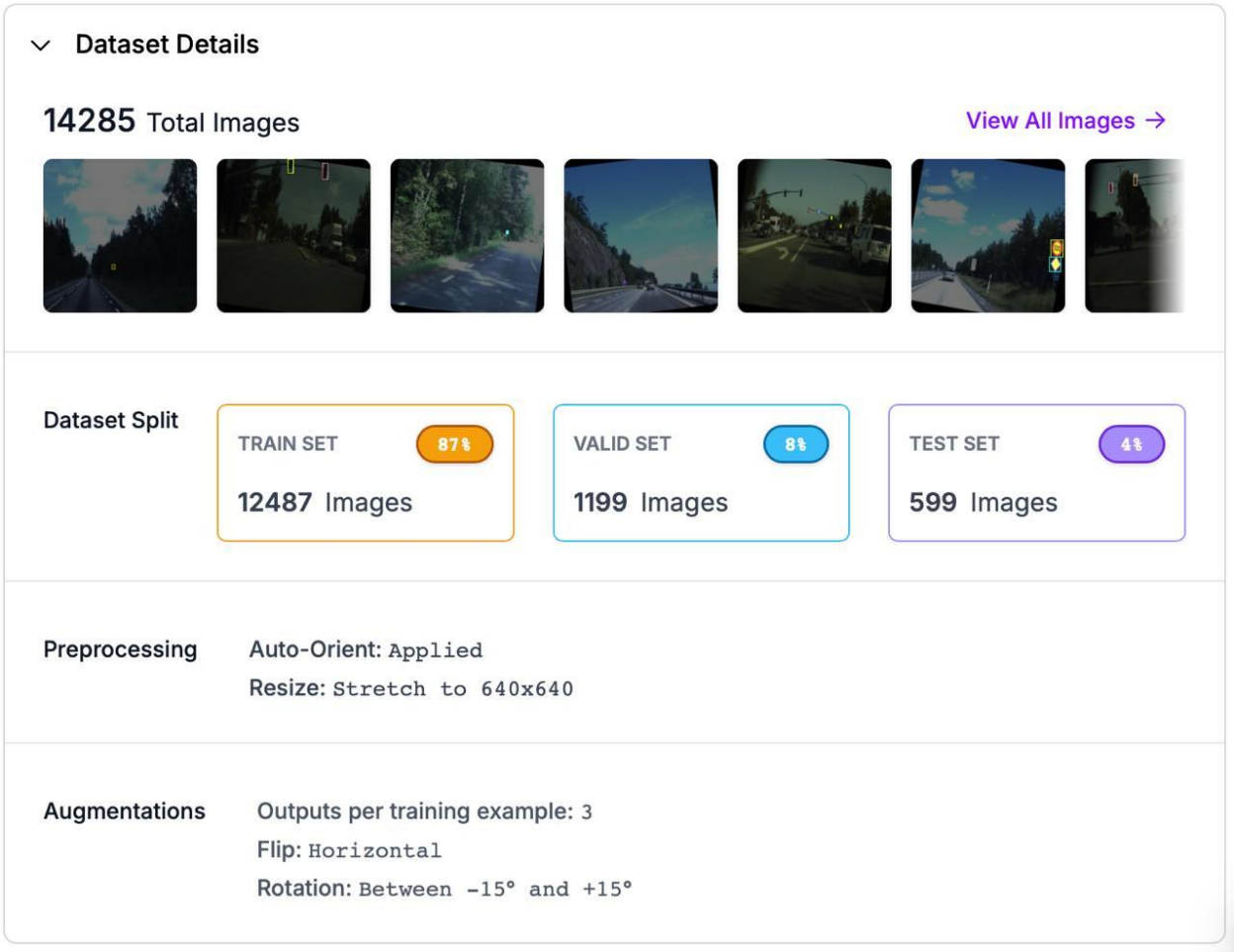



Photo 2

- This photo is associated with the dataset we are using for this model.

  Training set 87 percent
  Valid set 8 percent
  Test set 4 percent

**Results of section easy**







100_SIGN
100
PRIORITY_RO
40


120_SIGN
120


GIVE_WAY
OTHER
GIVE_WAY
OTHER


EATS
IDEAS
Green


Red
Red

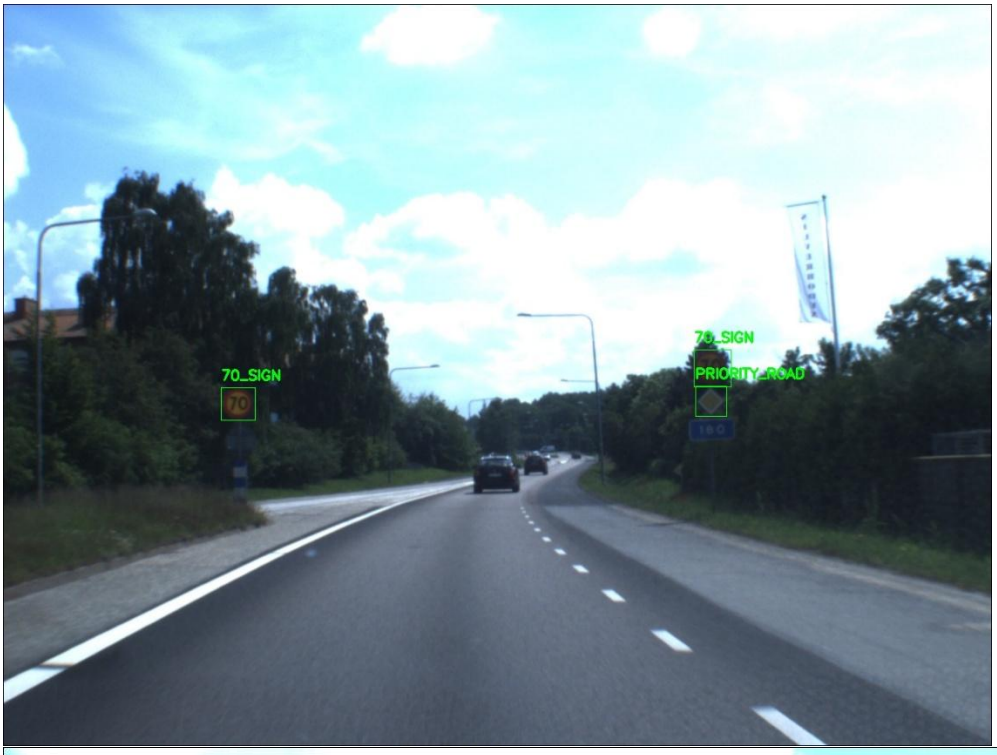

Green


Red
Red


Red
Red
Red
7AFR968


Green
Green

**Results of section hard**





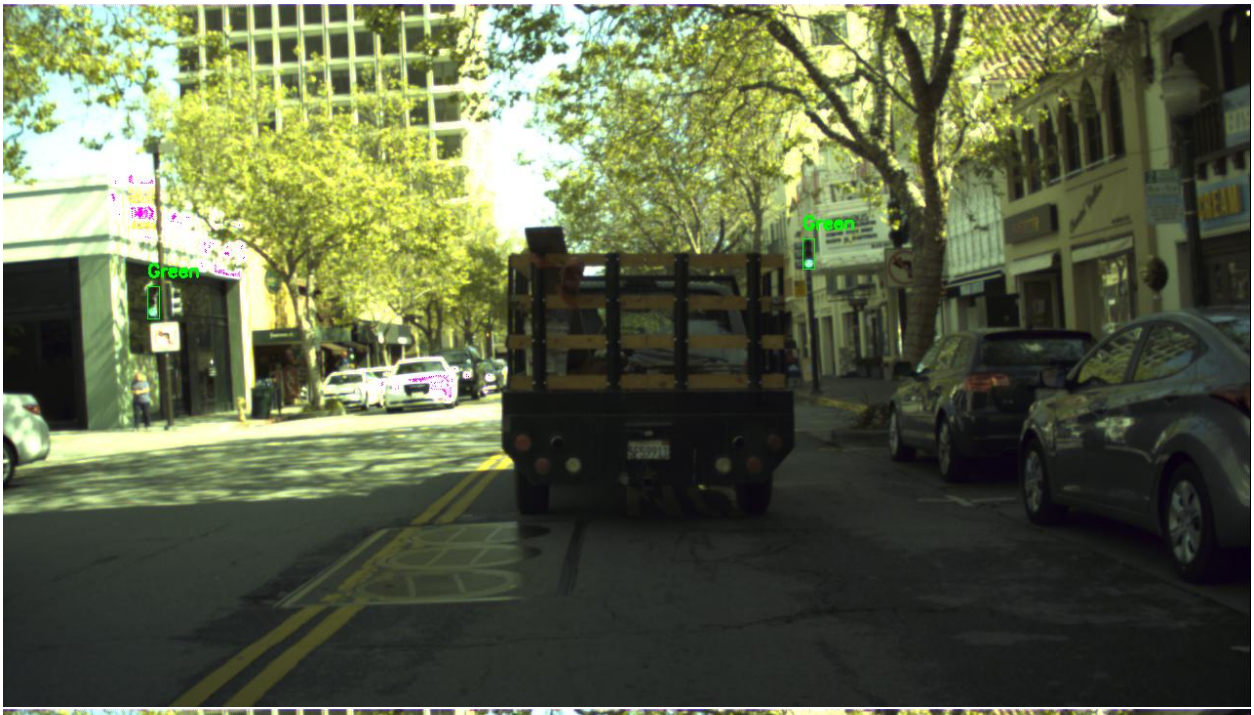


80_SIGN


50_SIGN
50
OTHER
GIVE_WAY
50_SIGN
50
JÖNKÖPING
Kasernvägen
Kompanigatan


Green


Red
Red
7AFR968


Green
Green


Green
Green